Evidence base for the DEL-v2 Technical Reference

The strongest empirical support for the Deterministic Engineering Loop comes from multi-agent role separation research and formal state machine governance, where peer-reviewed work directly validates core DEL-v2 claims. Several topics — particularly deterministic session recovery and structured handoff protocols — represent **genuine research gaps** where DEL-v2 would constitute novel prior art. The evidence gathered below spans 100+ sources across all seven topics, with quality distinctions (peer-reviewed, preprint, practitioner, vendor) marked throughout.

Topic 1: Formal state machine governance proves its case quantitatively

Published research directly supports the thesis that AI coding agents require formal state constraints. The evidence splits into two categories: frameworks that apply state machines to LLM workflows, and failure-mode data showing what happens without them.

Peer-reviewed and preprint frameworks applying FSMs to LLM agent workflows:

StateFlow (Wu et al., 2024, arXiv:2403.11322, Penn State / Microsoft Research) is the most directly applicable academic work. It formally models LLM task-solving as finite state machines with defined states, transitions, and output functions. (arXiv) (arxiv) On InterCode SQL and Bash benchmarks, StateFlow improved success rates while reducing costs by **4–6×** compared to baselines. (arXiv) The paper's core argument — that LLMs cannot reliably determine their own task-solving status, necessitating external FSM control (arXiv) — directly validates DEL-v2's governance model. (arxiv)

Automata-based constraints for language model decoding (ICLR 2025, peer-reviewed) uses finite state automata and pushdown automata to constrain LLM outputs to formal languages. Constraints compile **~7,000× faster** than prior work (Willard & Louf, 2023) and are provably correct by construction. (OpenReview) This provides the theoretical foundation for constraining agent outputs to valid tool calls within a given workflow state.

"The Fusion of Large Language Models and Formal Methods for Trustworthy AI Agents: A Roadmap" (arXiv:2412.06512, 2024) explicitly advocates bidirectional integration between LLMs and formal methods, implementing a PAT model checker agent using LangChain. (OpenReview) The paper argues that "relying solely on natural language reasoning" is insufficient for trustworthy agents. (OpenReview)

ESAA: Event Sourcing for Autonomous Agents (arXiv:2602.23193, 2026) proposes event sourcing with CQRS for LLM-based software engineering, where agents emit intentions validated by a deterministic orchestrator rather than directly writing state. (arXiv) It explicitly

critiques AutoGen, MetaGPT, and LangGraph for lacking "immutable audit trail and deterministic replay guarantees." (arXiv)

The **MCP Guardrail Framework** (IJIRMPS, 2025, published journal) integrates formal contract enforcement using Hoare-logic-style pre/postconditions in MCP tool manifests. (Ijirmps) It blocked **95–100% of unsafe or unauthorized actions** with only **8–13 ms** overhead per call. (Ijirmps)

Failure-mode evidence is stark. The Microsoft AI Red Team taxonomy (Bryan et al., 2024) identifies **12 failure modes with increased severity** in agentic contexts (microsoft) and explicitly recommends "safeguards that deterministically control the flow of an agentic AI system's execution." (Microsoft) A production voice agent (VOXAM, 2025) measured a **52% error rate** when LLMs controlled state transitions, including hallucinated transitions, stuck states, and instruction adherence degrading after turn 30+. (Voxam) The SWE-CI benchmark (Alibaba Research, March 2026) found **75% of tested models break previously working code** during long-term maintenance. (Noqta) SWE-Bench Pro (Deng et al., 2025, Scale AI) shows top models achieve only **~23% Pass@1** on real-world multi-file tasks. (Scale) (arXiv) An autonomous agent study (arXiv:2508.13143, 2025) found only **~50% task completion** across 34 tasks and 3 frameworks. (arXiv)

Anthropic's own data (February 2026) shows software engineering accounts for **~50% of agentic API activity**, (Anthropic) with 99.9th percentile session durations nearly doubling to >45 minutes. (anthropic) Their conclusion: "Effective oversight of agents will require new forms of post-deployment monitoring infrastructure." (Anthropic) (anthropic)

Industry frameworks implementing the pattern include LangGraph (models workflows as directed graphs (DesignRevision) with shared state; (DEV Community) the de facto standard), AutoGen with FSM integration (Wu et al., 2023, arXiv:2308.08155), (GitHub) Stately.ai's @statelyai/agent (XState + LLM decision-making), (GitHub) and the Auton Framework (arXiv:2602.23720, 2026) which introduces a "Constraint Manifold" where unsafe operations are "excluded by construction rather than detected after the fact." (arXiv)

"Institutional AI" (arXiv:2601.11369, 2026) compares ungoverned, constitutional (prompt-only), and institutional (runtime enforcement) governance of LLM agents, finding that **"prompt-level constraints are not binding"** — runtime enforcement consistently outperforms prompt-based governance. (arXiv) This directly validates DEL-v2's architectural enforcement over instruction-based constraints.

Key gap identified: No peer-reviewed paper applies formal state machine governance specifically to the software development lifecycle (requirements → implementation → testing → review → deployment). No process algebra (CSP, CCS, π-calculus) formalization exists for LLM agent coordination. No formal treatment of fail-closed/fail-open semantics exists specifically for AI agent systems, despite the concept being well-established in industrial safety (IEC 61508) and network security.

Topic 2: Cryptographic verification standards exist but not for AI-generated code specifically

The cryptographic integrity verification landscape is mature for general-purpose applications but has not been formally extended to AI-generated coding artifacts.

NIST standards provide the foundation. FIPS 180-4 (Secure Hash Standard, March 2012, DOI: 10.6028/NIST.FIPS.180-4) specifies SHA-256, which produces 256-bit digests enabling integrity determination with "very high probability" of detecting changes. NIST SP 800-107 Rev. 1 (Dang, August 2012, DOI: 10.6028/NIST.SP.800-107r1) provides security guidelines for hash applications, confirming SHA-256 offers **128 bits of collision resistance** and **256 bits of preimage resistance**. NIST is transitioning FIPS 180-4 to FIPS 180-5 (removing SHA-1). The NIST Cryptographic Algorithm Validation Program (CAVP) provides official test vectors and validation for SHA-256 implementations.

Tamper-evident logging has strong academic foundations. Crosby & Wallach (2009, USENIX Security, peer-reviewed) introduced history tree data structures using Merkle hash trees for tamper-evident logging with O(log n) proof sizes in append-only architectures. Usenix HARDLOG (Ahmad et al., 2022, IEEE S&P, peer-reviewed) enforces append-only policy via a hardware audit device that maintains monotonically-increasing indices, defending against adversaries with root privilege. Microsoft Nitro (arXiv:2509.03821, 2025) is the first tamper-evident logging system operating fully in eBPF. arXiv **AuditableLLM** (Li et al., 2026, *Electronics* 15(1):56, peer-reviewed) is the first unified audit framework specifically for LLMs, using **SHA-256 hash chaining** with **negligible utility degradation (<0.2% accuracy/macro-F1)**, 3.4 ms/step overhead (5.7% slowdown), and sub-second audit validation. MDPI Scilit

GPU-based hash verification (arXiv:2510.23938, 2025, Intel-affiliated) addresses a practical bottleneck: CPU-based SHA-256 verification of a typical **100 GB model requires several minutes**. GPU-native implementations using SYCL kernels achieve significant improvements, arXiv with hierarchical Merkle tree construction.

Reproducibility presents a fundamental challenge. "Generative AI Reproducibility and Consensus" (arXiv:2307.01898, 2023/2024) demonstrates that **identical runs of the same seed and prompt do not yield bit-identical results**, causing standard cryptographic hashes to diverge due to the avalanche effect. This means SHA-256 can verify AI workflow *state documents* and *artifacts* but cannot verify AI *output* reproducibility. Zero-Knowledge Proofs add ~10,000× computational overhead.

Supply chain security frameworks are mature but not AI-adapted. SLSA v1.1 (OpenSSF, April 2025) defines four progressive security levels for software artifact provenance using in-toto attestation format with cryptographic signatures. Sonatype research reports supply chain attacks increased **742% between 2019 and 2022**. Sigstore model-signing v1.0 (Maruseac et al.,

Google/NVIDIA/HiddenLayer, 2025) generates cryptographic hashes of ML models, signs them via OIDC, and stores signatures in the Rekor append-only transparency log. Sigstore Red Hat SPDX 3.0.1 includes formal AI and Dataset profiles (AI-BOM), while Sema Software's GBOM concept tracks whether code is "Pure GenAI" or "Blended." Semasoftware DVC uses content-addressable storage with hash-based versioning APXML as the de facto standard for ML data artifacts.

Critical gap: No peer-reviewed research studies SHA-256 verification integrated into AI-assisted code generation workflows end-to-end. No NIST guidance addresses cryptographic hash use specifically for AI/ML system artifacts. SLSA has not been formally adapted for AI-assisted development provenance. The question "which AI model, with what prompt, at what time, generated this code?" is not addressed in current provenance specifications.

Topic 3: Multi-agent role separation has the strongest peer-reviewed backing

This topic has the deepest evidence base, with multiple peer-reviewed papers at top venues directly supporting the auditor/executor separation pattern.

LLMs systematically favor their own outputs. Panickssery, Bowman & Feng (NeurIPS 2024, peer-reviewed, top-tier) found GPT-4 achieves **73.5% accuracy** at distinguishing its own outputs from others', with a **linear correlation between self-recognition capability and self-preference bias**. After fine-tuning on 500 examples, GPT-3.5 and Llama 2 achieve **>90% self-recognition accuracy**. All three models disproportionately favor their own summaries. neurips NeurIPS This is the single most important paper for arguing against AI self-review — it provides causal evidence that LLMs literally recognize and systematically favor their own work.

LLMs cannot self-correct reasoning. Huang et al. (ICLR 2024, peer-reviewed) demonstrate that performance sometimes **degrades** after self-correction attempts without external feedback. "The core issue is LLMs have trouble reliably evaluating the correctness of their own responses." arXiv Hugging Face Kamoi et al. (TACL, peer-reviewed, top-tier NLP journal) confirm that intrinsic self-correction "does not improve or even degrades performance" across arithmetic reasoning, QA, code generation, and plan generation. MIT Press

Sycophancy is pervasive and measurable. Sharma et al. (ICLR 2024, peer-reviewed, Anthropic) show all five state-of-the-art assistants exhibit sycophancy across four varied tasks, Anthropic OpenReview with "matching a user's views" being one of the most predictive features of human preference judgments. arXiv SycEval (Fanous et al., arXiv:2502.08177, 2025) measures sycophantic behavior in **58.19% of cases** across three leading models, with **regressive sycophancy** (leading to incorrect answers) in **14.66% of cases**. arXiv The ELEPHANT study (OpenReview, 2025) found LLMs preserve the user's face **45 percentage points more** than humans. OpenReview arXiv

LLM judges have measurable limitations. An ACM IUI 2025 study (peer-reviewed) found subject matter experts agreed with LLM judges only **68% of the time** in dietetics and **64% in mental health**, with LLM judges "often overlooking harmful or inaccurate aspects." (ACM Digital Library) Gu et al. (arXiv:2411.15594, 2025) catalog systematic biases: verbosity bias, position bias, self-enhancement bias, and authority bias. (arXiv) (arXiv)

Role separation demonstrably improves performance. "On the Resilience of LLM-Based Multi-Agent Collaboration with Faulty Agents" (OpenReview, peer-reviewed) found hierarchical structures with an independent "Inspector" reviewer role lost only **~5.5% accuracy** with faulty agents, while simple chain structures collapsed by **~23.7%**. Adding a Challenger/Inspector role recovered **up to 96% of lost performance**. (OpenReview) The MAST study (Cemri et al., arXiv:2503.13657, 2025, UC Berkeley) analyzed 200 conversation traces across 7 MAS frameworks, identifying **14 unique failure modes**. (UC Berkeley Sky Computin...) Adding high-level task verification to ChatDev yielded **+15.6% improvement** in task success. (arXiv) ChatDev achieves only **33.33% correctness** on ProgramDev without verification. (UC Berkeley Sky Computin...)

Foundational prior art for the pattern includes: AI Safety via Debate (Irving, Christiano & Amodei, arXiv:1805.00899, 2018, OpenAI, (OpenAI) 1000+ citations) — proposes adversarial separation between arguers and judges. Constitutional AI (Bai et al., arXiv:2212.08073, 2022, Anthropic) — uses principles as external constraints separating evaluation from generation. Constitutional Classifiers (Sharma et al., arXiv:2501.18837, 2025, Anthropic) reduce attack success rates from ~16% to **2%** using separate classifier models monitoring the main model. (arXiv) AgentReview (Jin et al., EMNLP 2024 Oral, peer-reviewed) (Agentreview) demonstrates **37.1% variation** in paper decisions due to reviewer biases in multi-agent peer review. (arXiv)

Gap: No published study specifically measures the effect of role separation on AI code review accuracy in a controlled experiment. No formal RBAC framework exists specifically for AI coding agent tool/action permissions. The sycophancy and self-preference findings have not been specifically replicated in code review contexts.

Topic 4: Session handoff is a practitioner-documented crisis with minimal academic study

Evidence for context persistence problems is overwhelming in practitioner sources but nearly absent from peer-reviewed literature.

How major tools handle context persistence:

- **Claude Code** starts each session with a fresh context window. (Claude) Persistence mechanisms: CLAUDE.md files (human-written, survive compaction) (Claude) and auto-memory (agent-written markdown at `~/.claude/projects/`). (Claude) (Anthropic) Sessions persist as `.jsonl` files. (Claude API Docs) Resume via `claude -c` or `claude -r <id>`.

Steve Kinney Official caveat: "Sessions persist the conversation, not the filesystem." Claude API Docs GitHub Issue #2954 reports developers "building 62-agent workaround systems" to handle context loss. GitHub

- **GitHub Copilot CLI** uses hierarchical memory: working memory (last 3 turns in full), compressed memory (LLM-generated summaries), and architectural memory (project structure). arXiv Auto-compaction at 95% token limit. ~20–30% of compacted tokens reserved for reasoning summaries. DeepWiki
- **Cursor** triggers summarization when context fills; agent retains references to history files. Cursor "After 20–30 exchanges, accumulated context can cause problems: the agent starts forgetting instructions." Developer Toolkit Notepads provide cross-session persistence. GitHub
- **Aider** has no native persistent session storage. Uses compressed git graph representation ("repo map") as context. AiderDesk community extension adds vector-based memory. GitHub
- **Windsurf** auto-generates memories during conversation, stored locally. Workspace-scoped (not cross-workspace). Windsurf Cascade memories are heuristic, not schema-validated.

Quantitative evidence on context degradation: Liu et al. (TACL 2024, peer-reviewed) found performance degrades arXiv by **>30%** when relevant information shifts to the middle of the context window, establishing the "lost-in-the-middle" effect even for long-context models. Neptune.ai Microsoft and Salesforce researchers found model performance dropped **39% on average** when transformed to multi-turn, with OpenAI's o3 model falling from **98.1% to 64.1% accuracy** (arXiv:2510.05381, 2025). Chroma Research (2025) confirms "across all models, significantly higher performance on focused prompts compared to full prompts." Trychroma AIMultiple benchmarking: "A model claiming 200k tokens typically becomes unreliable around 130k." AIMultiple

Context rot is well-documented in practice. The Stack Overflow 2025 Developer Survey found **developer trust in AI accuracy decreased from 43% in 2024 to 33%,** LogRocket with 45% reporting debugging AI-generated code takes more time than expected. LogRocket MindStudio (2025) defines "context rot" as gradual degradation where "suggestions don't suddenly become garbage — they just become slightly less accurate." MindStudio One practitioner analysis found agents lose CLAUDE.md instructions, forget modified files, and repeat completed work DEV Community after ~45 minutes of accumulated context.

Recovery-Bench (Letta, 2025) is the closest benchmark to session recovery evaluation. Key finding: "the best AI models today are often distracted or misled by failed trajectories in their context." Letta **Claude 4 Sonnet drops in rank under recovery conditions while GPT-5 moves ahead**, showing that resilience to context pollution is not correlated with raw problem-solving strength. Letta

MemGPT (Packer et al., ICLR 2024, peer-reviewed) introduces OS-inspired virtual context management with hierarchical memory tiers. (arXiv +2) Now evolved into the Letta framework. (Leonie Monigatti) **Mem0** (Chhikara et al., arXiv:2504.19413, 2025) achieves **91% reduction in p95 latency** and 26% improvement on LOCOMO benchmark versus OpenAI's approach. "Beyond the Context Window" (arXiv:2603.04814, 2026) found long-context GPT-5-mini achieves higher factual recall than memory systems but at linearly scaling cost — memory systems become more economical after approximately **10 interactions**. (arXiv)

Critical gaps: No peer-reviewed study directly measures session restoration fidelity. No comparative study of file-based vs. memory-based vs. embedding-based approaches exists specifically for coding contexts. No study quantifies exactly what percentage of architectural decisions or debugging state survives a session handoff. The academic memory survey (arXiv:2603.07670, 2026) confirms: "The field still lacks a community-standard evaluation harness." (arXiv)

Topic 5: Automated tests systematically miss visual failures, with strong evidence

The gap between automated test results and real-browser behavior is documented across academic research, industry studies, and browser vendor sources.

The flaky test problem is extensively quantified. Luo et al. (FSE 2014, peer-reviewed, ACM SIGSOFT) established the foundational taxonomy: **45% async wait**, **20% concurrency**, **12% test order dependency**. Google's testing data (Micco, 2016) reveals **~16% of all tests** have some level of flakiness, (Talent500) (googleblog) with **84% of observed pass-to-fail transitions** involving a flaky test. (TestDino) (googleblog) Large tests are flaky **14% of the time** per week. (Trunk) DeFlaker (Bell et al., ICSE 2018, peer-reviewed) found DeFlaker achieves 95.5% recall versus Maven's 23%. (ACM Digital Library) Google's Ziftci & Cavalcanti (ICSME 2020, peer-reviewed) achieved 82% accuracy in locating flaky test root causes. (IEEE Xplore) Atlassian reports **150,000+ hours of developer time wasted per year** on flaky test reruns, with flaky tests causing 21% of master build failures. (Atlassian) Slack measured flaky tests at **56.76% of CI failures** before remediation. (Autonoma AI) Bitrise reports teams experiencing flakiness grew from **10% in 2022 to 26% in 2025**. (TestDino) Romano et al. (SBES 2023, peer-reviewed) found **race condition, logic issues, and test dependency** are the main causes of UI-specific flaky tests. (ACM Other conferences)

Six categories of UI bugs systematically pass functional test suites (documented by Applitools, TestMetry, and multiple practitioner sources): layout shifts, element overlap, color/contrast failures, responsive breakpoint failures, broken/missing images, and font rendering changes. All pass because DOM elements exist and function — they are simply not visually correct. (TestMetry) The Southwest Airlines case is the canonical example: a "Buy Now" button hidden under a Terms overlay, invisible and unclickable, with all functional tests passing on a site generating

~$2.5M/hour. TestMetry Applitools demonstrates that "identical DOM structures can have different visual output" and "different DOM structures can render identically." InfoQ

Headless browser testing has measurable rendering gaps. BrowserStack documentation confirms "headless Chrome renders elements differently" with specific limitations in timing/race conditions, browser features gated behind visible UI, and third-party scripts. BrowserStack Chromium's `chromium-headless-shell` differs from the full browser binary. BrowserStack Chrome 137+ (2025–2026) **deprecated and removed** the automatic SwiftShader fallback for WebGL in headless mode, breaking CI pipelines. CopyProgramming Default headless WebGL performance: **8 fps versus 60 fps expected** (createIT). createIT Firefox's Bugzilla #1375585 documents that WebGL was completely disabled in headless mode due to crash issues, Mozilla Bugzilla with a developer noting in 2024: "This is still a pretty big hole in our testing on firefox since we don't have any CI coverage for graphics tests." Mozilla Bugzilla

React Three Fiber has fundamental pointer event divergence from DOM semantics. R3F pointer events use THREE.Raycaster for hit testing Zabbix rather than DOM hit testing. `onPointerLeave` and `onPointerEnter` fire in **opposite order** compared to native DOM events. GitHub `setPointerCapture` "doesn't behave at all like the DOM equivalent" (acknowledged as bug by maintainers). GitHub Standard browser automation tools dispatch DOM events, which may not trigger the same behavior as real user interactions with R3F's custom event system.

Accessibility testing coverage has a measurable ceiling. Deque Systems (2021, industry study analyzing 2,000+ audits) found automated tools can test only **15 of 50 WCAG 2.1 AA criteria (~30%)**; by volume, **57.38% of issues** are found by automation, meaning **at least 43% require manual testing**. Vervali WebAIM Million (2025) found 94.8% of top million home pages had WCAG failures, averaging 51 errors per page. WebAIM

Peer-reviewed academic work includes Leotta et al. (ICWE 2014) comparing visual vs. DOM-based web locators empirically across six applications, and Roy Choudhary et al. (ICSM 2010) demonstrating that neither DOM analysis nor visual comparison alone suffices for cross-browser testing.

Gaps: No peer-reviewed study quantifies the rate at which "all tests green, UI visually broken" occurs in production. No academic study compares headless vs. headed test outcomes for the same suite at scale. No peer-reviewed work addresses R3F or WebGL testing in headless browsers. The "false confidence" problem (tests that reliably pass while missing real bugs) has minimal academic coverage.

Topic 6: Regulatory requirements are converging on append-only logging

The EU AI Act, NIST, and ISO 42001 create a convergent regulatory framework that effectively mandates tamper-evident, append-only logging for AI systems.

EU AI Act Article 12 (Regulation 2024/1689, enacted June 2024, high-risk requirements enforceable August 2026) requires: "High-risk AI systems shall technically allow for the automatic recording of events (logs) over the lifetime of the system." Europa Logs must enable risk detection (Art. 79), post-market monitoring (Art. 72), and operational oversight (Art. 26(5)). EU Artificial Intelligence Act VDE **Article 19** mandates providers **keep** these logs for **at least 6 months**, EU Artificial Intelligence Act Europa with penalties up to **€30 million or 6% of global annual turnover**. Critically, Article 12 mandates tamper-evident logging but **does not specify cryptographic mechanisms** Veritaschain — this is a regulatory gap that organizations implementing hash chains or Merkle trees would fill proactively.

NIST SP 800-92 (Kent & Souppaya, 2006, DOI: 10.6028/NIST.SP.800-92) provides foundational enterprise log management guidance NIST CSRC covering generation, transmission, storage, access, and disposal. NIST CSRC It requires protecting "the confidentiality, integrity, and availability of logs." NIST Revision 1 (initial public draft, 2023) is informed by OMB M-21-31 (post-SolarWinds) and introduces hot/warm/cold data categories. NIST CSRC **Neither version addresses AI/ML system logging specifically.**

ISO/IEC 42001:2023 (published December 2023), the world's first AI management system standard, ISO includes **Control A.6.2.8: AI System Recording of Event Logs**, requiring logs that are "immutable, traceable, and instantly exportable" with "provenance and policy-aligned retention." ISMS.online Emergent Mind The AWS Security Blog (2025) maps ISO 42001 lifecycle stages to STRIDE threat modeling, identifying **repudiation risks** from "lack of model decision logs" at the verification stage. AWS Microsoft 365 Copilot has achieved ISO 42001 certification. Microsoft Learn

OpenTelemetry GenAI Semantic Conventions (CNCF, experimental/development status) define structured telemetry for AI agents including `gen_ai.agent.name`, `gen_ai.agent.id`, OpenTelemetry `gen_ai.tool.name`, `gen_ai.tool.call.parameters`, and `gen_ai.tool.call.result`. GitHub Contributors include IBM, Google, Microsoft, Amazon, and Elastic. OpenTelemetry OpenLLMetry (Traceloop, Apache 2.0) extends OTel for AI model observability. GitHub Datadog supports OTel GenAI conventions v1.37+. Datadog **However, OTel itself provides no tamper-evidence or append-only guarantees** — it is an instrumentation layer, not a storage guarantee.

AI-specific tamper-evident logging is emerging. AuditableLLM (Li et al., 2026, *Electronics*, peer-reviewed) provides SHA-256 hash chaining with **<0.2% utility degradation** and sub-second validation. MDPI Scilit AgentLens (open-source) implements SHA-256 hash chains per session for AI agents. GitHub Transparency.dev/Trillian (Google, open-source) provides Merkle tree-backed append-only ledgers used in Certificate Transparency. Transparency ScienceDirect ISO/IEC DIS 24970:2025 (draft) proposes an AI-specific logging standard VDE but has been criticized as "necessary but insufficient" for addressing internal manipulation.

Audit log manipulation is a documented threat pattern. MITRE CAPEC-268 defines the attack pattern, mapping to ATT&CK T1070 (Indicator Removal). (mitre) HiddenLayer (2025) reports **74% of organizations** had an AI breach in 2024, with **45% opting not to report** due to reputational concerns. Only **32% deploy a technology solution** for AI threats. (HiddenLayer) PCAOB analysis (Dharmasiri et al., 2022, *Journal of Business Ethics*, peer-reviewed) found manipulation of audit evidence results in significantly higher financial penalties. (Springer)

Gap: No published case studies document audit log manipulation specifically in AI/ML systems. No unified standard exists for tamper-evident AI logging. OpenTelemetry GenAI conventions remain experimental. (OpenTelemetry) EU AI Act implementation guidelines for Articles 12/19 (covering log granularity, format, and retention specifications) are still pending. (Dataiku)

Topic 7: Deterministic session recovery is a genuine research gap — DEL-v2 is novel

This topic yielded the most significant finding for the whitepaper: **there is zero peer-reviewed work on deterministic state recovery for AI coding sessions.** DEL-v2's structured HANDOFF.md protocol constitutes genuinely novel prior art.

The closest existing systems address workflow determinism, not cognitive determinism. Temporal.io separates deterministic workflow logic from non-deterministic LLM calls. After a crash, workflow replays from event history — Activity results are recovered from logs, not re-executed. (Wayland Zhang) OpenAI's Codex agent and Replit Agent 3 are built on Temporal. (Temporal) LangGraph saves checkpoints at every super-step boundary, (Langchain) supporting "time travel" replay, (GitHub) but nodes that re-execute may produce different LLM results. (DEV Community +2) Replit's Snapshot Engine uses Copy-on-Write block-level storage for instant filesystem forks with git commits. (Replit)

Structured context documents measurably improve LLM initialization. AIMultiple's benchmark (2025, practitioner) found structured context ("GroundedCtx") delivered **+13.0% improvement in task accuracy** for Gemini 2.5 Flash and **+5.4% for Claude Sonnet 4** compared to unstructured text. (AIMultiple) Liu et al. (TACL 2024, peer-reviewed) established that information at the **beginning** of context receives strongest attention (Neptune.ai) (arXiv) — providing direct theoretical support for placing a structured HANDOFF.md at the start of a new session's context. Microsoft documentation confirms "LLMs are adept at parsing and interpreting structured formats like JSON or YAML." The FACTS Grounding benchmark (Google DeepMind, 2024) validates that LLMs grounded with structured documents achieve measurable improvements in factual accuracy.

The only working implementation of a validated handoff protocol is the rjmurillo/ai-agents open-source project (2026, practitioner evidence), which implements HANDOFF.md as read-only during sessions (enforced by ADR-014), with JSON schema validation, pre-commit hooks

verifying no changes during session timeframe, and session chaining through `relatedSessions` fields. DeepWiki Other practitioner implementations include the Session Handoff Protocol for Claude Code Blakelink (Blake Link, 2025) and the CONTINUITY MCP Server (open-source, 8 tools for session persistence including `handoff_quality`). GitHub

Commercial API state guarantees are limited and opaque. OpenAI's Conversations API provides persistent objects with durable identifiers and no 30-day TTL, with a compaction endpoint that shrinks context while preserving "latent understanding." But there is **no guarantee of semantic fidelity** of compacted representations and no formal verification of state correctness. Anthropic's Memory Tool (beta since September 2025) is client-side and file-based GitHub using Markdown files. Internal evaluation shows **39% improvement** combining memory with context editing and **84% token reduction** in 100-turn evaluations. But memory quality depends on LLM judgment — there is no schema enforcement, no verification of completeness, and no deterministic guarantee.

All existing memory systems use probabilistic retrieval. Mem0 uses vector embeddings, MemGPT uses LLM self-managed paging, arXiv Claude Memory uses LLM-decided saves, GitHub OpenAI uses opaque compaction. The academic survey "Hindsight is 20/20" (arXiv:2512.12818, December 2025) confirms current systems "treat memory as an external layer that extracts salient snippets from conversations, stores them in vector or graph-based stores, and retrieves top-k items into the prompt of an otherwise stateless model." arXiv

Six critical gaps constitute the whitepaper's novel contribution space:

1. **No peer-reviewed work on deterministic session recovery for AI coding.** The closest work (Temporal, LangGraph) addresses workflow determinism, not reasoning state determinism. DEL-v2's structured HANDOFF.md protocol is genuinely novel.
2. **No comparative study of structured vs. unstructured handoff documents.** The AIMultiple benchmark provides suggestive evidence (+5.4–13.0% from structured context) but no study specifically compares schema-validated handoff files against unstructured dumps for coding session recovery.
3. **No formal definition of "state fidelity" for AI sessions.** Neither OpenAI nor Anthropic define or guarantee state fidelity. No established metric exists for measuring whether a recovered session has the same "understanding." DEL-v2 could propose such a metric.
4. **No benchmark for cross-session state recovery in coding tasks.** Existing benchmarks (LOCOMO, LongMemEval, Recovery-Bench) focus on conversational memory, not coding task continuity.
5. **The probabilistic/deterministic boundary is unresolved.** True deterministic recovery of LLM cognitive state may be fundamentally impossible given model non-determinism. The whitepaper should distinguish between deterministic recovery of *artifacts and context*

documents (achievable, verifiable via SHA-256) and deterministic recovery of *LLM reasoning state* (approximable but not guaranteeable).

6. **No industry standard for AI agent handoff protocols.** Multiple implementations exist but with no standardization. DEL-v2's HANDOFF.md schema could contribute toward establishing one.

Conclusion: where DEL-v2 stands on evidence

The evidence landscape reveals a clear pattern. **Three topics have strong peer-reviewed support:** multi-agent role separation (NeurIPS 2024, ICLR 2024, TACL, EMNLP 2024), formal state machine governance (StateFlow, automata-based decoding at ICLR 2025, multiple failure-mode studies), and cryptographic integrity verification (NIST FIPS 180-4, Crosby & Wallach 2009, AuditableLLM 2026). **Two topics have strong regulatory/standards backing:** append-only audit logging (EU AI Act Articles 12/19, ISO 42001 A.6.2.8, NIST SP 800-92) and supply chain security (SLSA v1.1, Sigstore, SPDX 3.0.1). **Two topics are primarily practitioner-evidenced with significant research gaps:** session handoff (Recovery-Bench and context rot documentation but no fidelity benchmarks) and UI testing insufficiency (Google flaky test data at 16%, accessibility coverage ceiling at 57%, but no controlled studies of false confidence rates).

The most strategically important finding is that **deterministic session recovery is a genuine research gap**. DEL-v2's protocol — structured HANDOFF.md with defined schema, SHA-256 verification, append-only logging, and role-separated validation — has no direct peer-reviewed predecessor. The whitepaper should position this explicitly as novel contribution while anchoring its component claims (formal governance, cryptographic integrity, role separation, append-only logging) in the substantial existing evidence base documented above.